

제약 조건 기반 퍼징 하네스 생성

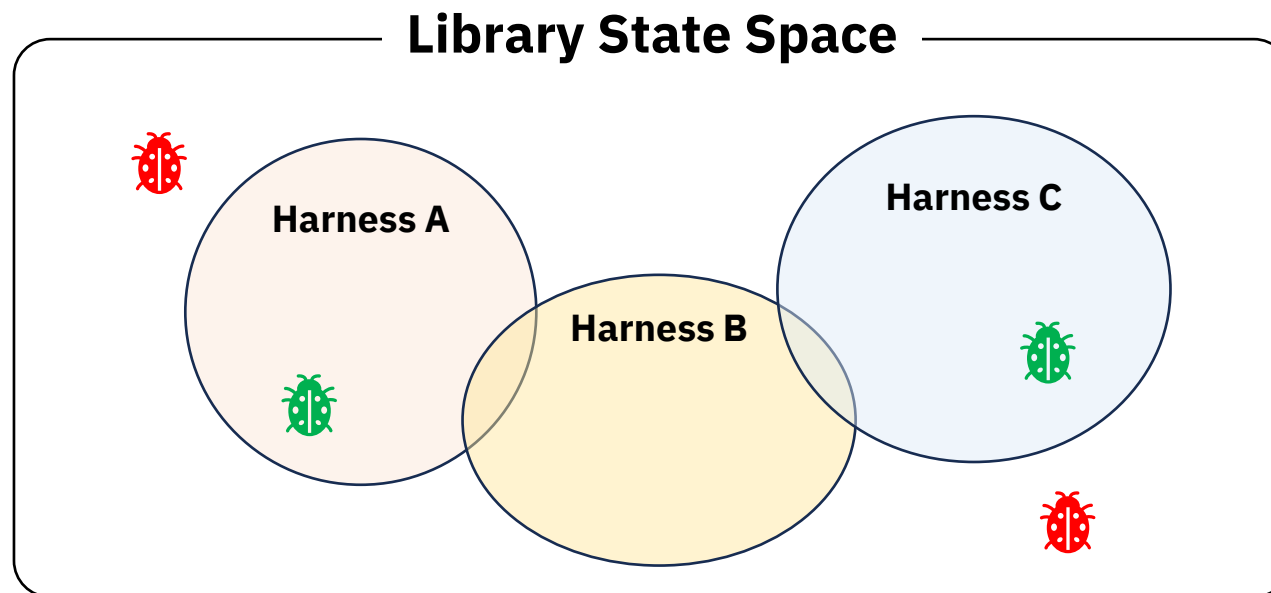
양종문, 김운호

한양대학교 소프트웨어공학연구실



배경 및 동기

- 라이브러리 API를 퍼징하기 위해서는 진입점인 퍼징 하네스가 필요함.
 - 하네스는 라이브러리에서 도달 가능한 상태 공간의 경계를 정의함.
 - 상태 공간이 버그를 포함하지 않으면 퍼징 시간과 무관하게 버그를 탐지할 수 없음.**



배경 및 동기

- OSS-Fuzz는 구글이 2016년 시작한 오픈소스 대상 연속 퍼징 인프라임.
- OSS-Fuzz에 따르면 수백만 CPU 시간을 퍼징에 소요하였음에도 커버리지는 30% 수준.
 - 즉, 프로젝트의 70% 부분에서 취약점 탐지에 실패함.
 - 원인은 퍼징 엔진이 아니라 기존 타깃(하네스)의 결함.

Additionally, the main challenge for most integrated OSS-Fuzz projects is ensuring high code coverage. Most OSS-Fuzz projects have fairly low runtime coverage (~30%) despite millions of hours of CPU time. This means we are not finding vulnerabilities in approximately 70% of each project that we're fuzzing. Our preliminary research found that many fuzz blockers (as determined by FuzzIntrospector) are because of deficiencies in existing targets, rather than deficiencies in fuzzing engines.

OSS-Fuzz 블로그의 Background 중 일부^[1]

[1] https://google.github.io/oss-fuzz/research/llms/target_generation/

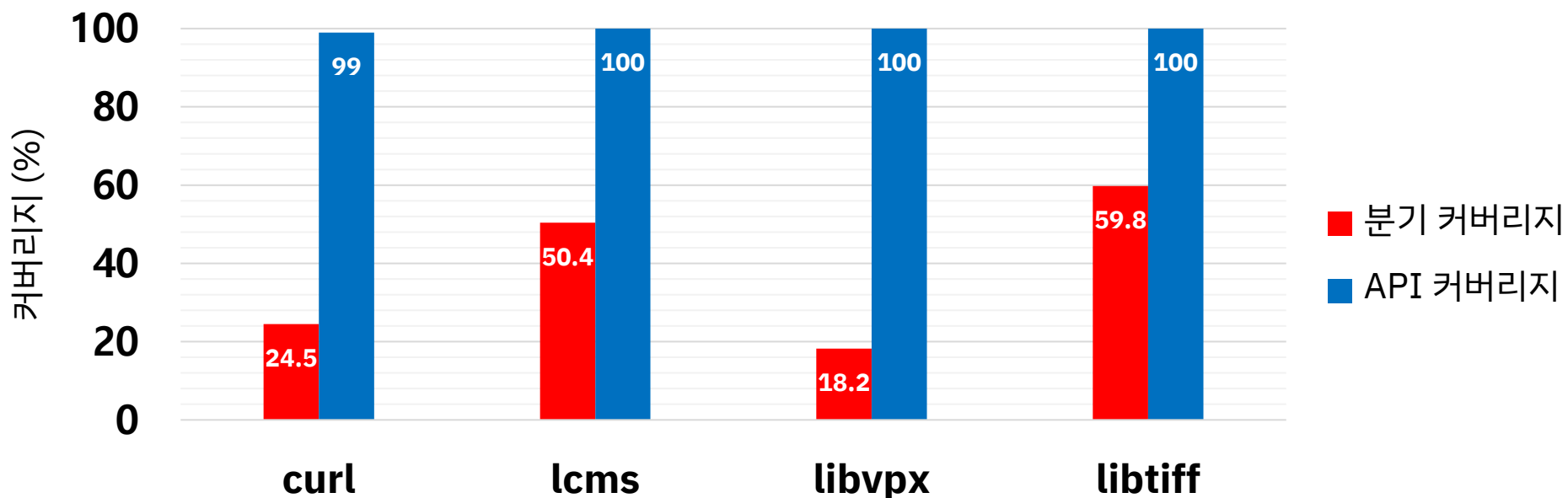
배경 및 동기

- 라이브러리를 커버하기 위해 퍼징 하네스를 자동으로 만드려는 다양한 연구가 진행됨.
- **프로그램 분석 기반**
 - 정적 분석, 소비자 코드 등 코드에서 추출한 **구조 정보**로 하네스를 조립.
 - 한계: API 사용 제약을 이해하지 못해 **오용 및 false positive** 다수.
- **LLM 기반**
 - 헤더, 주석, API 문서 등 **자연어 의미 정보**까지 이해 → 오용 없이 실행되는 하네스 생성.
 - 한계: **유효한 호출 ≠ 상태 도달**, 여전히 도달하지 못하는 상태 공간이 많음.

배경 및 동기

- 그럼에도 여전히 도달하지 못하는 상태 공간이 존재함.
- 최신 LLM 기반 하네스 생성 프레임워크 PromeFuzz도 많은 분기를 커버하지 못함.

*API 커버리지: 전체 public API 중 생성된 하네스 코드에 포함된 API 비율



PromeFuzz 논문 상의 실험 데이터^[2]

[2] Liu et al., "PromeFuzz: A Knowledge-Driven Approach to Fuzzing Harness Generation with LLMs," CCS 2025.

배경 및 동기

- 기존 기법의 목표는 "유효하게 실행되는 하네스"이므로 도달하지 못한 상태 공간이 남음.
- 미도달 상태에 도달하기 위해 API 시퀀스, 인자, 입력 제약 조건 등을 만족할 필요가 있음.
 - 그러나 상태 공간은 매우 넓기 때문에 미도달 상태 전부를 보완하는 것은 비현실적임.
 - 가치가 있는 상태 공간을 선별할 필요가 있음.
 - 즉, **버그 발생 가능성이 높은 상태 식별**이 필요.

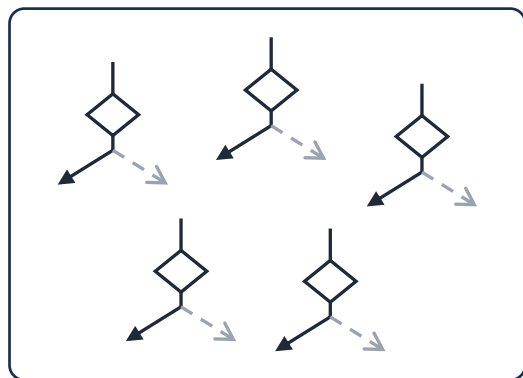
도전 과제

1. 버그 가능성이 높은 상태를 어떻게 식별할 것인가?
2. 해당 상태가 가진 제약 조건을 어떻게 충족할 것인가?

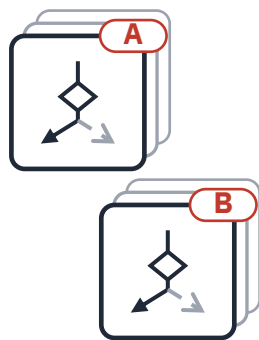
해결 방안

1. 버그 가능성이 높은 상태를 어떻게 식별할 것인가?

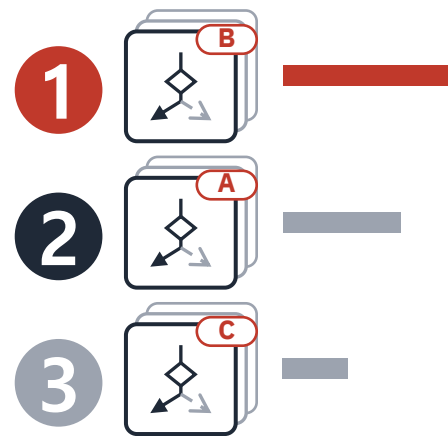
- 버그 가능성이 높은 상태: 메모리 연산이 실행되는, 도달하지 못한 프로그램 상태.
 - 미도달 분기는 수천 개이므로 각각에 대한 분석·생성·검증 비용이 큼.
- 미도달 분기에 대해 **군집화 수행** 후 **우선순위를 부여**하여 버그 가능성을 식별.



분기 집합



분기 묶음

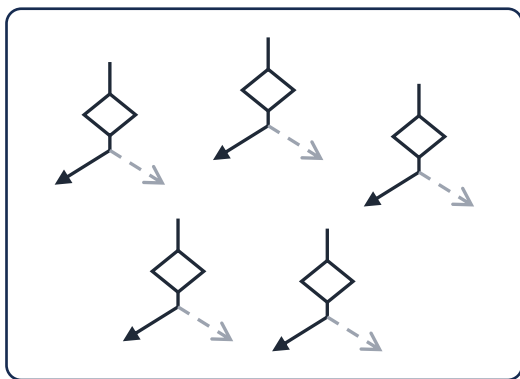


묶음 우선순위

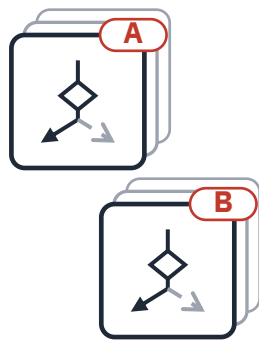
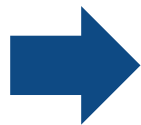
해결 방안

1. 버그 가능성이 높은 상태를 어떻게 식별할 것인가? - **군집화**

- 같은 변수를 사용하는 분기라면 변이 시 다른 분기에 비해 커버할 확률이 높을 것임.
- **따라서 수천 개 분기 각각이 아닌 동일 함수, 동일 변수를 하나로 묶어 다룸.**



미도달 분기 집합



분기 묶음

**동일한 함수 내에서
동일한 변수를 사용하는
분기를 군집화.**

해결 방안

1. 버그 가능성이 높은 상태를 어떻게 식별할 것인가? - 우선순위 부여

- 해당 분기를 커버하면 버그로 이어질 확률이 높은 것을 고르는 것이 목표.
- **한정된 예산 내에 많은 상태를 탐색하기 위해 세 가지 지표를 사용함.**

① 위험도

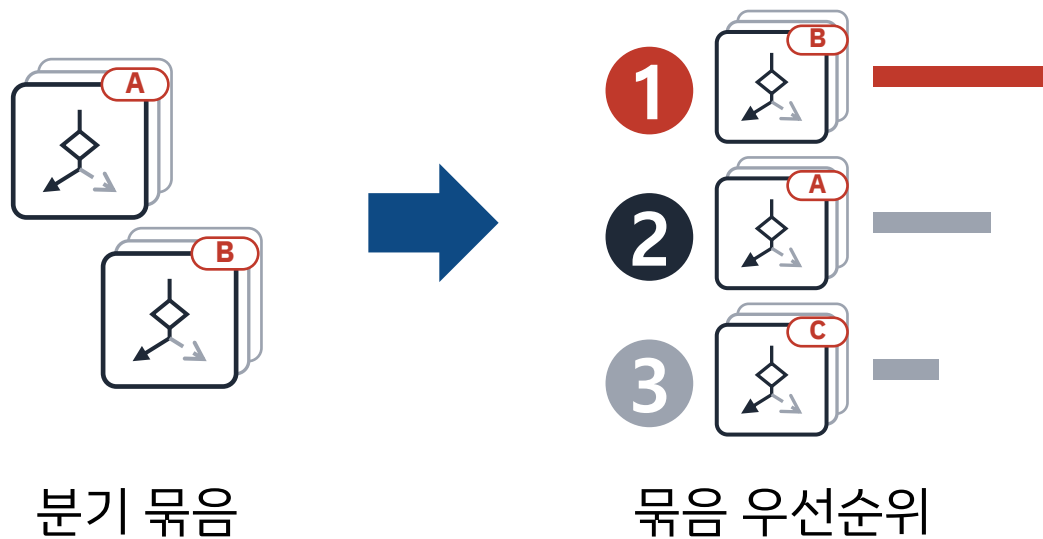
그 안에 버그가 있어야 함.

② 파급력

하나로 여러 상태를 열어야 함.

③ 난이도

실제로 탐색할 수 있어야 함.



해결 방안

1. 버그 가능성이 높은 상태를 어떻게 식별할 것인가? - 우선순위 부여

- 아래 세 값을 taint 분석 및 가중합 하여 높은 순으로 하네스 개선을 수행함.

① 위험도 (위험한 메모리 연산에 쓰이는가?)

- 해당 분기 집합이 접근한 배열 인덱스 접근, 메모리 복사, 곱셈 연산 등의 **메모리 연산 수**를 측정.

② 파급력 (얼마나 많은 미도달 분기와 얽혀 있는가?)

- 해당 분기 집합이 가지고 있는 **하위 미도달 분기의 합**을 측정.

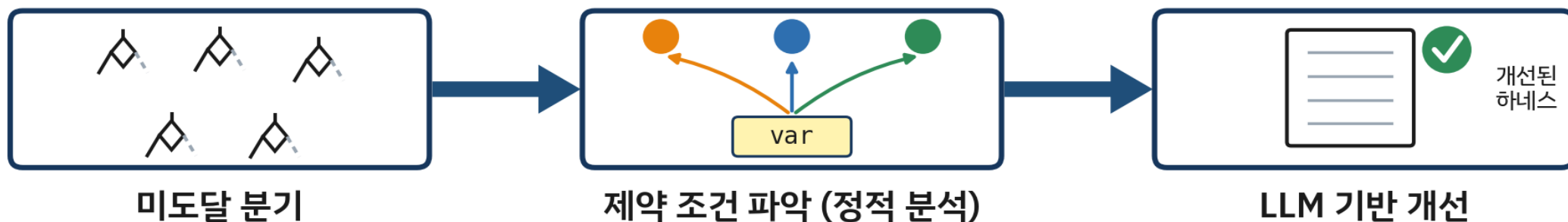
③ 난이도 (하네스를 개선하기 위한 난이도가 어떻게 되는가?)

- 해당 분기 집합이 가지고 있는 분기의 **깊이 평균**을 측정.

해결 방안

2. 해당 상태가 가진 제약 조건을 어떻게 충족할 것인가?

- 분기의 변수가 특정 값이어야 제약 조건을 만족하며 미도달 상태를 커버 가능함.
 - 기존 하네스는 유효한 호출에 편향되어, 특정 값을 만족하지 못하여 상태 미도달.
 - 미도달 분기를 정적 분석으로 역추적하여 제약 조건을 파악하고 LLM을 통해 개선.



해결 방안

2. 해당 상태가 가진 제약 조건을 어떻게 충족할 것인가?

- 분기 내 변수의 값을 하네스 레벨에서 세팅할 수 있는 방법은 세 가지가 있음.

① 인자 상수 입력

```
1 mul(640, 480);
```

② 세터 API

```
1 setICC(h, icc, len);
```

③ 입력 파서

```
1 parse_chunk(file);
```

→ 미도달 분기의 변수를 정적 분석으로 역추적하여 어떤 방식으로 세팅 되는지 확인.

→ 세팅 주체에 따라 다른 개선 기법 적용.

해결 방안

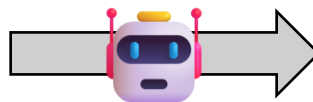
2. 해당 상태가 가진 제약 조건을 어떻게 충족할 것인가?

→ 분기에 영향을 주는 제약을 출처에 따라 분류하고 LLM을 통해 하네스 개선 수행.

인자 개방

세터가 **인자 상수**인 경우

```
1 mul(640, 480);
```



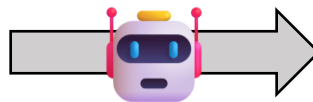
```
1 mul(fuzz1(), fuzz2());
```

분기 및 API 인자 상수 위치
+ 상수→퍼저 바이트 노출 지시

하네스 재구성

세터가 **다른 API**인 경우

```
1 decode(h, jpeg);
```



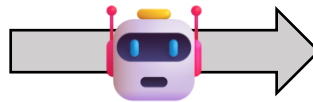
```
1 setICC(h, icc, len);  
2 decode(h, jpeg);
```

세터 API 목록·호출 순서
+ 세터 호출 삽입 지시

입력 합성

세터가 **입력 파서**인 경우

```
seed["a.jpeg"]
```

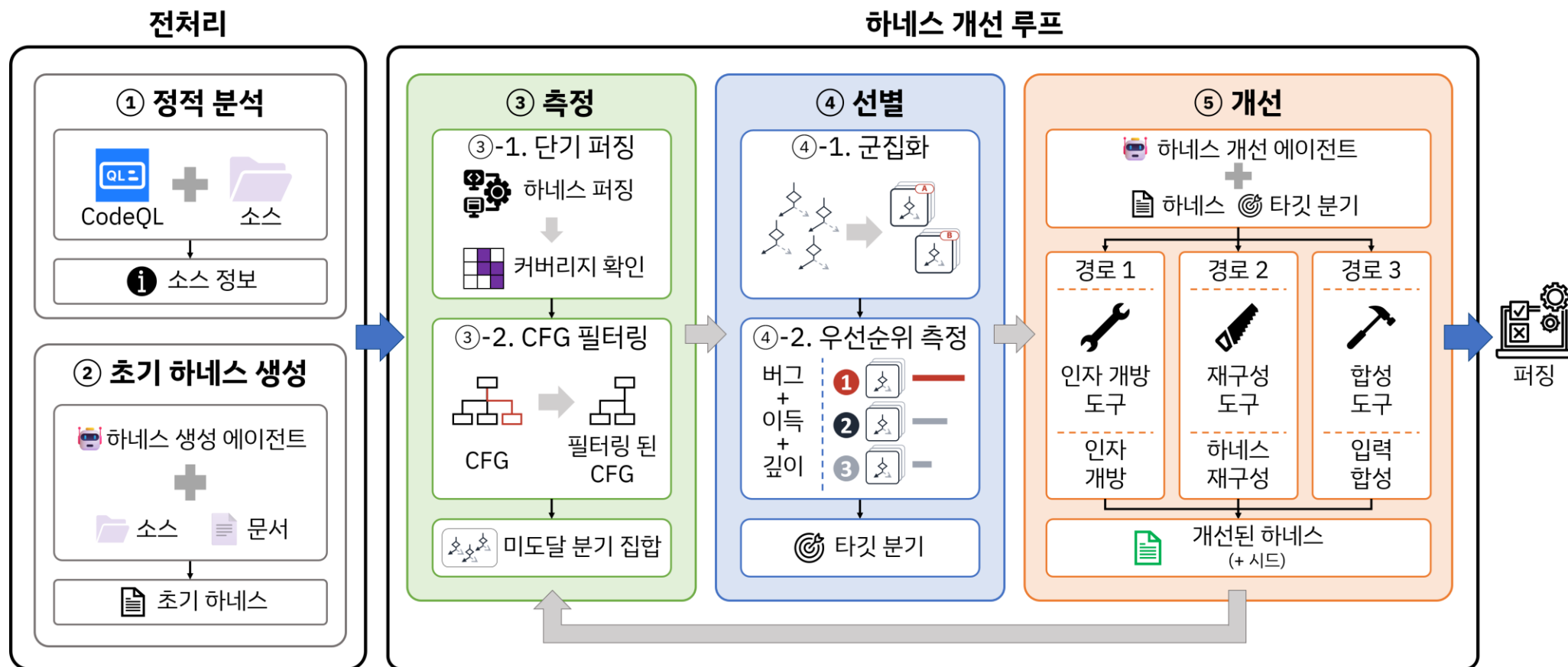


```
seed["a.jpeg", "b.png"]
```

목표 특성을 포함한 입력 합성 지시
+ encode API로 입력 합성 지시

해결 방안

- 중요해 보이는 분기를 우선순위를 매겨 성격에 따라 하네스를 점진적으로 개선하자!



배경 및 동기

- 라이브러리 API 퍼징을 위해서는 하네스가 필요하며 하네스의 상태 공간이 버그를 포함하지 않으면 퍼징 시간과 무관하게 버그를 탐지할 수 없음.
- 기존 퍼징 하네스 자동 생성 기법의 목표는 구문론적·의미론적 정보를 활용하여 "유용하게 실행되는 하네스"를 만드는 것임.
→ **유용한 호출 ≠ 상태 도달**이며, 유용한 호출은 상태 도달을 보장하지 않음.
도달하지 못한 특정 상태 공간에 도달하기 위해서는 다양한 API 인자, API 시퀀스, 입력 제약 조건을 만족해야 함.
→ **상태 공간은 너무 넓다는 문제가 있어, 가치 있는 상태 공간을 선별할 필요가 있음.**
- 상태 공간을 유의미하게 탐색하기 위해서는 **버그 가능성이 높은 상태를 식별**하고, 해당 상태에 **도달하기 위한 제약 조건을 반영**하는 것이 관건임.
→ Challenge 1. 버그 가능성이 높은 상태를 어떻게 식별할 것인가?
→ Challenge 2. 해당 상태에 도달하기 위한 제약 조건을 어떻게 충족할 것인가?

해결 방법 1: 군집화 및 우선순위 부여

군집화

- 미도달 분기는 매우 많아 분석 비용이 크므로 효율을 위해 군집화.
- 동일한 함수 내의 동일한 변수를 사용하는** 분기를 군집화.

우선순위 부여

- 한정된 예산 내에 많은 상태를 탐색하기 위해 세 가지 지표 사용.
- 위험도** (위험한 메모리 연산에 사용되는가?)
 - 해당 분기 집합이 접근한 메모리 관련 연산 수를 측정.
- 파급력** (얼마나 많은 미도달 분기와 얽혀있는가?)
 - 해당 분기 집합이 가지고 있는 하위 미도달 분기의 합을 측정.
- 난이도** (하네스를 개선하기 위한 난이도가 어떻게 되는가?)
 - 해당 분기 집합이 가지고 있는 분기의 길이 평균을 측정.

<pre>image_width ├─w*bpp ├─memcpy(.,.,w*n) ├─arr[w*row] └─malloc(w*h)</pre>	<pre>bits ├─memcpy(.,.,bits) └─arr[bits]</pre>
4개의 메모리 관련 연산 → 점수 ↑	2개의 메모리 관련 연산 → 점수 ↓

Taint 분석 결과 비교

해결 방법 2: 제약 조건 개선 LLM 에이전트

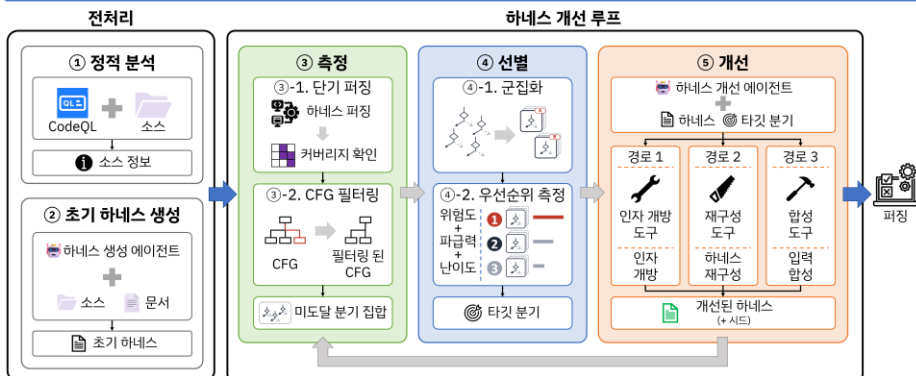
하네스 개선 에이전트

- 세터에 따른 분기의 제약 조건 유형을 판별하여 LLM 기반 개선 수행.
- 경로 1. **세터가 인자 상수인 경우.**
 - 하네스가 인자를 상수로 고정하여 내부 분기 탐색이 불가함.
 - 판별: 해당 변수가 API 인자 변경에 영향받는지 확인.
 - 해결: 인자를 퍼져 입력으로 노출하거나 범위 내 무작위로 설정.

```
tj3Set(h, TJPARAM_LOSSLESS, 0); // 상수 고정
+ tj3Set(h, TJPARAM_LOSSLESS, fuzz()); // 퍼져 노출
tj3Compress8(h, ...);
```
- 경로 2. **세터가 다른 API인 경우.**
 - 함수가 호출되지 않거나, 선행 API로 상태가 설정되지 않음.
 - 판별: 해당 함수 호출 여부, 변수의 write API 호출 여부를 확인.
 - 해결: Setter API를 호출 시퀀스에 추가하여 하네스를 구성.

```
+ tj3SetICCPProfile(h, icc, len); // Setter API 호출
tj3Decompress8(h, jpeg, out);
```
- 경로 3. **세터가 입력 파서인 경우.**
 - 하네스에서 jpeg 파일을 요구하나, 시드에 png 밖에 없음.
 - 판별: 입력 파서 API를 통해서 변수가 직접 설정되는지 확인.
 - 해결: LLM이 조건을 만족하는 입력을 판단·합성.

조건 기반 퍼징 하네스 생성



향후 연구 계획

- 다양한 우선순위 스코어링 기법 고안 및 다양한 라이브러리에서 확인.
- 하네스 개선 에이전트의 개선 성공률 및 평가 지표 정립.
- 하네스 개선 단계의 수렴 판정 기준 정립.
- 분기 묶음 내에서 개선 대상이 될 타겟 분기 선택 기준 정립.